Artificial Intelligence

AI-2002

Lab 08

Adversarial Search

National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2002	Artificial Intelligence Lab

Contents

1. Objectives

1. An Introduction to Game theory and Adversarial Search Algorithm.
2. Tasks

2. Introduction to Game Theory

Game Theory is a branch of **mathematics** used to model the **strategic interaction** between different players in a context with predefined rules and outcomes. According to game theory, a game is played between two players. To complete the game, one has to win the game, and the other loses automatically.

Figure 01: **Adversarial Search**

Adversarial Search is a **Game-Playing Technique** where the agents are surrounded by a competitive environment. A conflicting goal is given to the agents (multiagent). These agents compete with one another and try to defeat one another in order to win the game. Such conflicting goals give rise to the adversarial search. Here, game-playing means discussing those games where human intelligence and logic factor is used, excluding other factors such as luck factor. **Tic-tac-toe**, **Chess**, **Checkers**, etc., are such types of games where no luck factor works, only the mind work

2.1 Elements of Game Playing Search

To play a game, we use a **Game Tree** to know all the possible choices and to pick the best one out. So, It is the initial state from which a game begins.

There are the following elements of game-playing:

Player(s): It defines which player is having the current turn to make a move in the state.

Actions(s): It defines the set of legal moves to be used in a state.

Result(s, a): It is a transition model which defines the result of a move.

Terminal-Test(s): It defines that the game has ended and returns true.

Utility(s, p): It defines the final value with which the game has ended. This function is also known as the Objective function or Payoff function. The price which the winner will get i.e.

- **-1:** If the Player **loses**.
- **+1:** If the Player **wins**.
- **0:** If there is a draw between the Players.

For example, in chess, tic-tac-toe, we have two or three possible outcomes. Either to win, to lose, or to draw the match with values +1,-1, or 0.

Let's understand the working of the elements with the help of a game tree designed for tic-tac-toe.

Here, the node represents the game state and the edges represent the moves taken by the players.

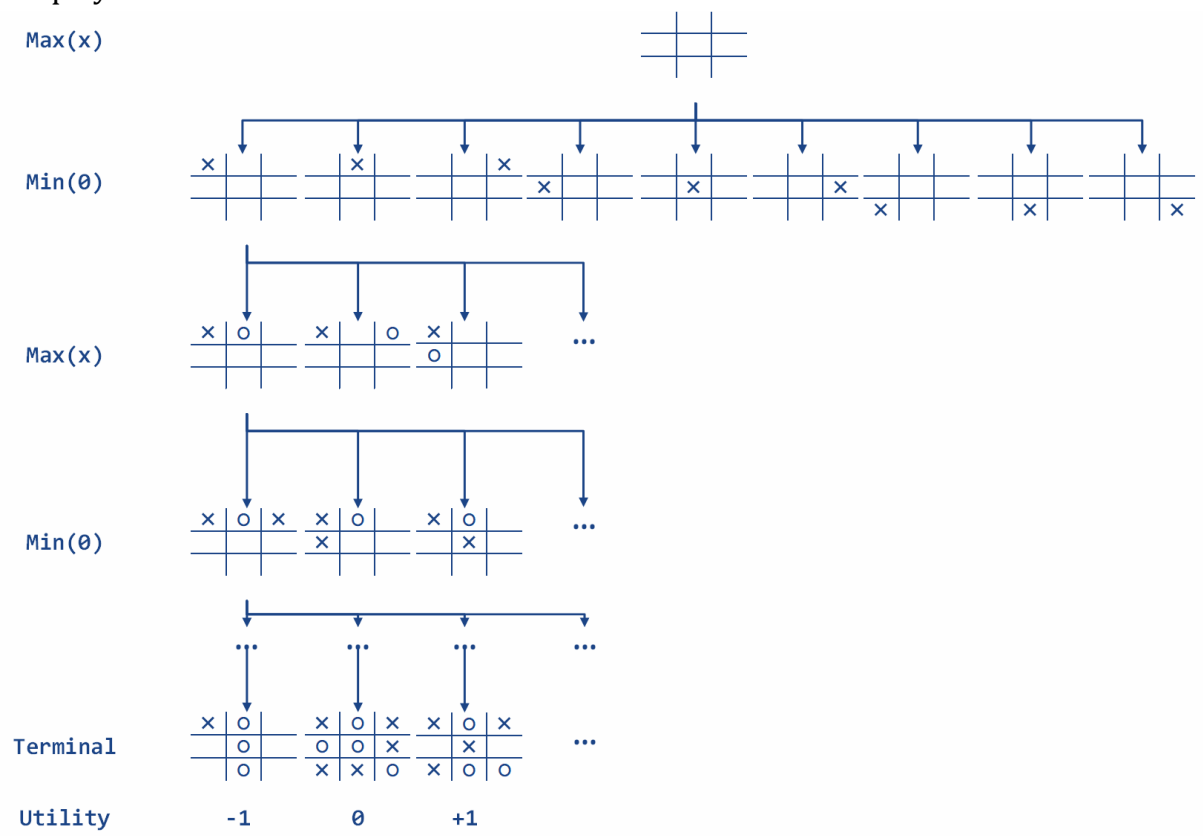


Figure 02: **A Game Tree for Tic-Tac-Toe**

Initial State (S_o): The top node in the game tree represents the initial state in the tree and shows all the possible choices to pick out one.

Player(s): There are two players, **Max** and **Min**. **Max** begins the game by picking one best move and placing **X** in the empty square box.

Actions(s): Both the players can make moves in the empty boxes chance by chance.

Results(s, a): The moves made by **Min** and **Max** will decide the outcome of the game.

Terminal-Test(s): When all the empty boxes will be filled, it will be the terminating state of the game.

Utility: At the end, we will get to know who wins: **Max** or **Min**, and accordingly, the price will be given to them.

2.2 Types of Algorithms in Adversarial Search

Unlike Normal Search, in Adversarial search, the result depends on the players which will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and under limited time.

2.2.1 MiniMax Algorithm

Minimax is a decision-making strategy, which is used to minimize the losing chances in a game and to maximize the winning chances. This strategy is also known as 'Minmax'. It is a two-player game strategy where if one wins, the other loses the game. We can easily understand this strategy via a game tree where the nodes represent the states of the game and the edges represent the moves made by the players in the game. Players will be two namely:

Min: Decrease the chances of winning the game.
Max: Increases his chances of winning the game.

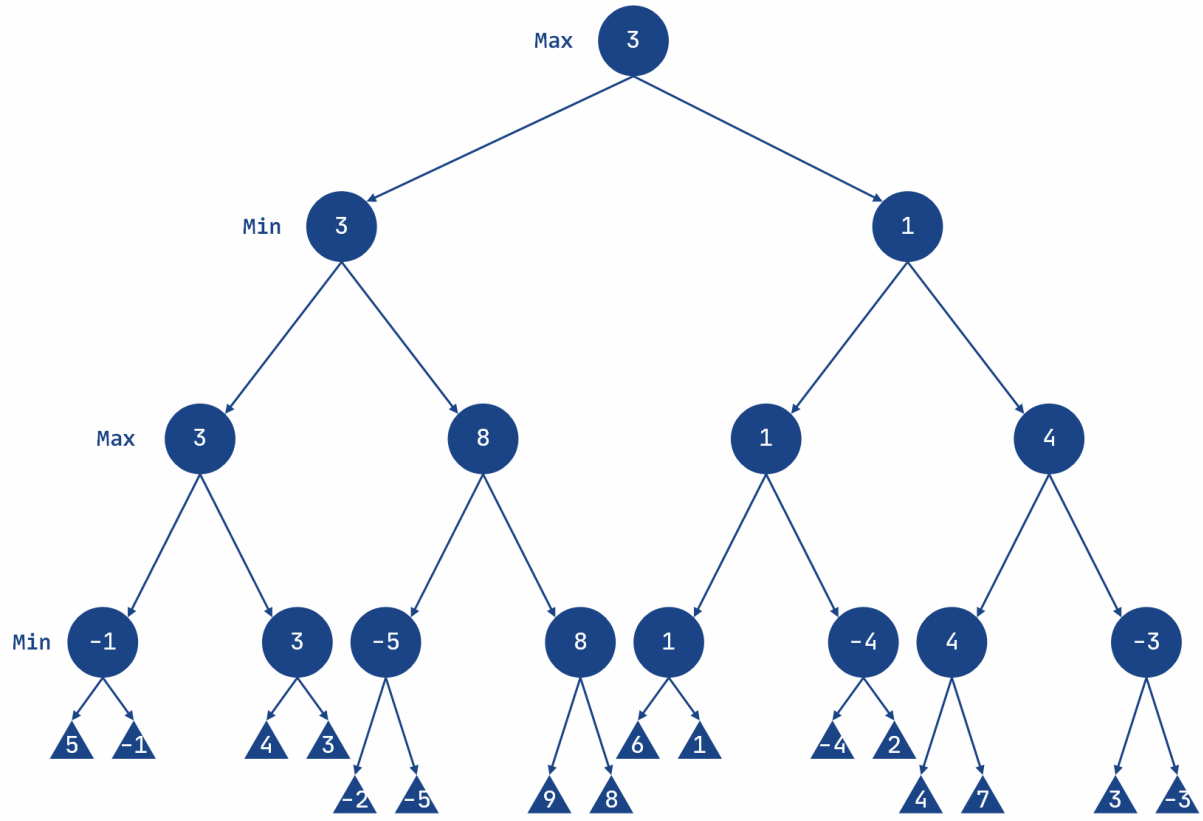
In the minimax strategy, the result of the game or the utility value is generated by a heuristic function, which propagates from the leaf nodes back to the root node using backtracking. At each node, the algorithm selects the maximum or minimum value depending on whether it is the Max or Min player's turn.

The **Minimax** strategy follows the **Depth-First Search (DFS)** concept. In this approach, there are two players, **Min** and **Max**, who take turns alternately during the game. For instance, when **Max** makes a move, the next turn belongs to **Min**. Once **Max** makes a move, it is fixed and cannot be changed.

The **Minimax** strategy follows the **Depth-First Search (DFS)** concept to traverse the game tree. In this approach, two players, **Min** and **Max**, alternate turns during the game. For example, when **Max** makes a move, the next turn belongs to **Min**. During the evaluation of a specific path, the move made by **Max** is treated as fixed within that path. **However**, the algorithm **backtracks** to explore other paths and determine the optimal move. The DFS approach ensures each path is explored fully before backtracking, making it more suitable than **Breadth-First Search (BFS)** for this algorithm.

Flow of Minimax Algorithm:

Keep on generating the game tree/ search tree till a limit.
Compute the move using a heuristic function.
Propagate the values from the leaf node to the current position following the minimax strategy.



Players in the Game:

Two players: **Max** and **Min**.
They alternate turns throughout the game.

Max's Turn:

Max starts the game by selecting a path and exploring (propagating) all the nodes along that path.
After exploring the nodes, **Max** backtracks to the initial node.
Max then selects the best path, i.e., the one where his **utility value is maximized**.

Min's Turn:

After **Max** completes, it's **Min's** turn.
Min explores paths and backtracks afterward, selecting the path that minimizes **Max's** chances of winning by choosing the minimum utility value among its successors.
However, Min chooses the path that **minimizes Max's chances of winning** or reduces the utility value.

Node Value Selection:

If the current level is a **minimizing level** (**Min's** turn), the node accepts the **minimum value** from its successor nodes.
If the current level is a **maximizing level** (**Max's** turn), the node accepts the **maximum value** from its successor nodes.

Key Concept:

Max tries to maximize the utility value.
Min tries to minimize **Max's** chances by reducing the utility value.

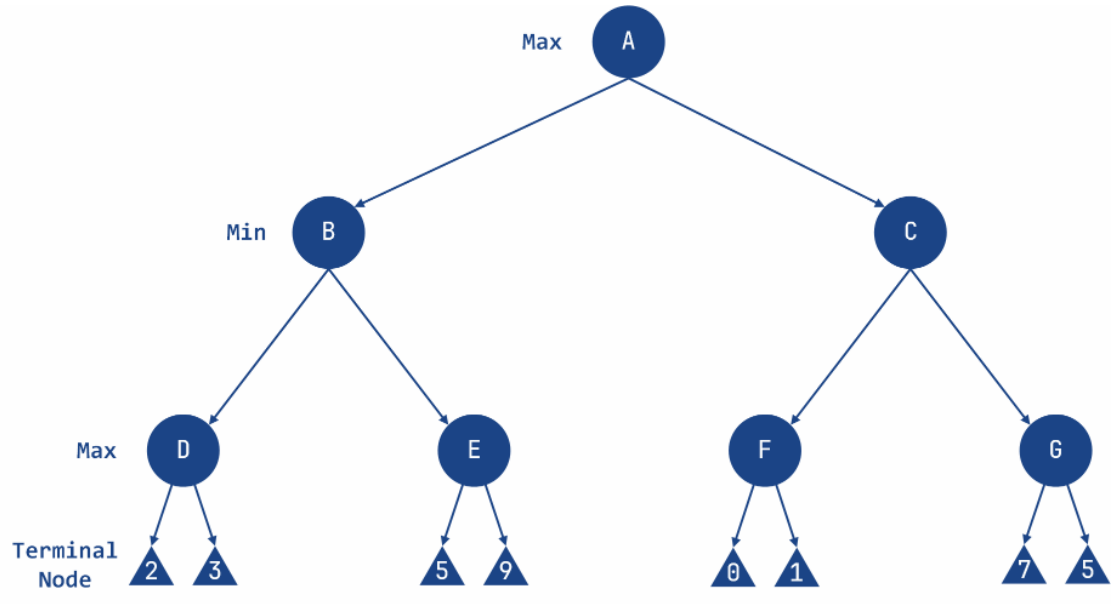
2.2.1.1 Pseudo Code for MiniMax Algorithm

```
function minimax(node, depth, maximizingPlayer) is
    if depth == 0 or node is a terminal node then
        return static evaluation of node

    if MaximizingPlayer then // for Maximizer Player
        maxEva = -infinity
        for each child of node do
            eva = minimax(child, depth - 1, false)
            maxEva = max(maxEva, eva) // Gives the maximum of the values
        return maxEva
    else // for Minimizer Player
        minEva = +infinity
        for each child of node do
            eva = minimax(child, depth - 1, true)
```

```
            minEva = min(minEva, eva) // Gives the minimum of the values
        return minEva
```

2.2.1.2 MiniMax Algorithm - Example




```
import math

class Node:
    def __init__(self, value=None):
        self.value = value
        self.children = []
        self.minmax_value = None

class MinimaxAgent:
    def __init__(self, depth):
        self.depth = depth

    def formulate_goal(self, node):
        # in Minimax, the goal is to compute the minimax value for the root
node
        return "Goal reached" if node.minmax_value is not None else
"Searching"

    def act(self, node, environment):
```

```
        goal_status = self.formulate_goal(node)
        if goal_status == "Goal reached":
            return f"Minimax value for root node: {node.minmax_value}"
        else:
            return environment.compute_minimax(node, self.depth)

class Environment:
    def __init__(self, tree):
        self.tree = tree
        self.computed_nodes = []

    def get_percept(self, node):
        return node

    def compute_minimax(self, node, depth, maximizing_player=True):
        if depth == 0 or not node.children:
            self.computed_nodes.append(node.value)
            return node.value

        if maximizing_player:
            value = -math.inf
            for child in node.children:
                child_value = self.compute_minimax(child, depth - 1, False)
                value = max(value, child_value)
            node.minmax_value = value
            self.computed_nodes.append(node.value)
            return value
        else:
            value = math.inf
            for child in node.children:
                child_value = self.compute_minimax(child, depth - 1, True)
                value = min(value, child_value)
            node.minmax_value = value
            self.computed_nodes.append(node.value)
            return value

def run_agent(agent, environment, start_node):
    percept = environment.get_percept(start_node)
    agent.act(percept, environment)
```

```
# sample tree
root = Node('A')
n1 = Node('B')
n2 = Node('C')
root.children = [n1, n2]

n3 = Node('D')
n4 = Node('E')
n5 = Node('F')
n6 = Node('G')
n1.children = [n3, n4]
n2.children = [n5, n6]

n7 = Node(2)
n8 = Node(3)
n9 = Node(5)
n10 = Node(9)
n3.children = [n7, n8]
n4.children = [n9, n10]

n11 = Node(0)
n12 = Node(1)
n13 = Node(7)
n14 = Node(5)
n5.children = [n11, n12]
n6.children = [n13, n14]

# define depth for Minimax
depth = 3

agent = MinimaxAgent(depth)
environment = Environment(root)
run_agent(agent, environment, root)
print("Computed Nodes:", environment.computed_nodes)

print("Minimax values:")
print("A:", root.minmax_value)
print("B:", n1.minmax_value)
print("C:", n2.minmax_value)
print("D:", n3.minmax_value)
```

```
print("E:", n4.minmax_value)
print("F:", n5.minmax_value)
print("G:", n6.minmax_value)
```

Output

```
Computed Nodes: [2, 3, 'D', 5, 9, 'E', 'B', 0, 1, 'F', 7, 5, 'G', 'C', 'A']
Minimax values:
A: 3
B: 3
C: 1
D: 3
E: 9
F: 1
G: 7
```

2.2.2 Alpha-Beta Pruning

Alpha-Beta Pruning is an optimized version of the **Minimax** algorithm. One major drawback of the **Minimax** strategy is that it explores every node in the game tree, which can be time-consuming and computationally expensive. **Alpha-Beta Pruning** addresses this issue by reducing the number of nodes explored in the search tree.

The key idea behind alpha-beta pruning is to "**prune**" or eliminate branches of the tree that do not affect the final decision. This allows the algorithm to focus only on relevant paths while still making the same decisions as the **Minimax** algorithm. By using this technique, unnecessary calculations are avoided, improving efficiency.

Alpha-Beta Pruning works based on two threshold values:

1. **Alpha (-∞):** The best value that the **Max** player can guarantee at any point. It acts as the lower bound (negative infinity initially).
2. **Beta (+∞):** The best value that the **Min** player can guarantee at any point. It acts as the upper bound (positive infinity initially).

This **pruning** mechanism ensures that branches that cannot influence the outcome are skipped, making the algorithm faster without compromising accuracy.

2.2.2.1 Pseudo Code for Alpha-Beta Pruning Algorithm

```
function minimax(node, depth, alpha, beta, maximizingPlayer) is
    if depth == 0 or node is a terminal node then
        return static evaluation of node

    if MaximizingPlayer then // for Maximizer Player
        maxEva = -infinity
        for each child of node do
            eva = minimax(child, depth - 1, alpha, beta, false)
            maxEva = max(maxEva, eva)
            alpha = max(alpha, maxEva)
            if beta <= alpha then
                break
        return maxEva
    else // for Minimizer Player
        minEva = +infinity
        for each child of node do
            eva = minimax(child, depth - 1, alpha, beta, true)
            minEva = min(minEva, eva)
            beta = min(beta, eva)
            if beta <= alpha then
                break
        return minEva
```

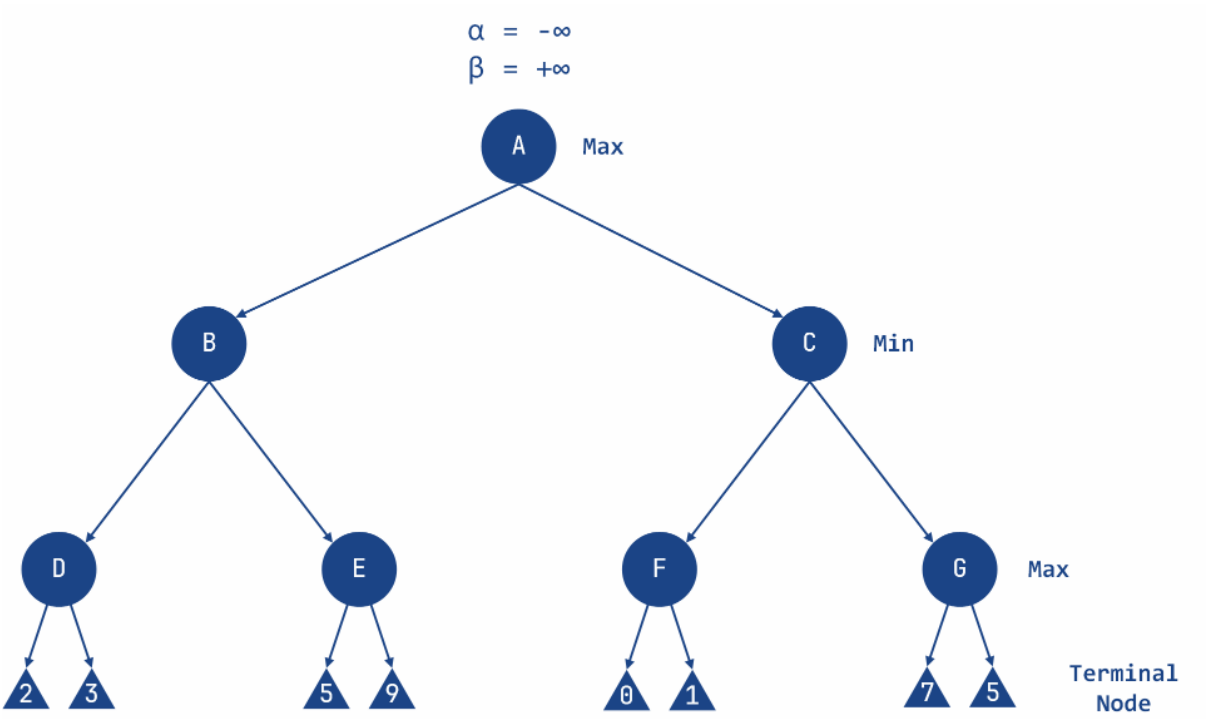
2.2.2.2 Working of Alpha-Beta Pruning Algorithm

Let's take an example of a two-player search tree to understand the working of Alpha-beta pruning.

Step 1: Initialize the Algorithm

- The **Max** player starts at **Node A**.

- Initialize alpha (α) and beta (β) values:
  - α = -∞ (worst case for **Max**)
  - β = +∞ (worst case for **Min**)
- Pass these values to the next node (**Node B**).

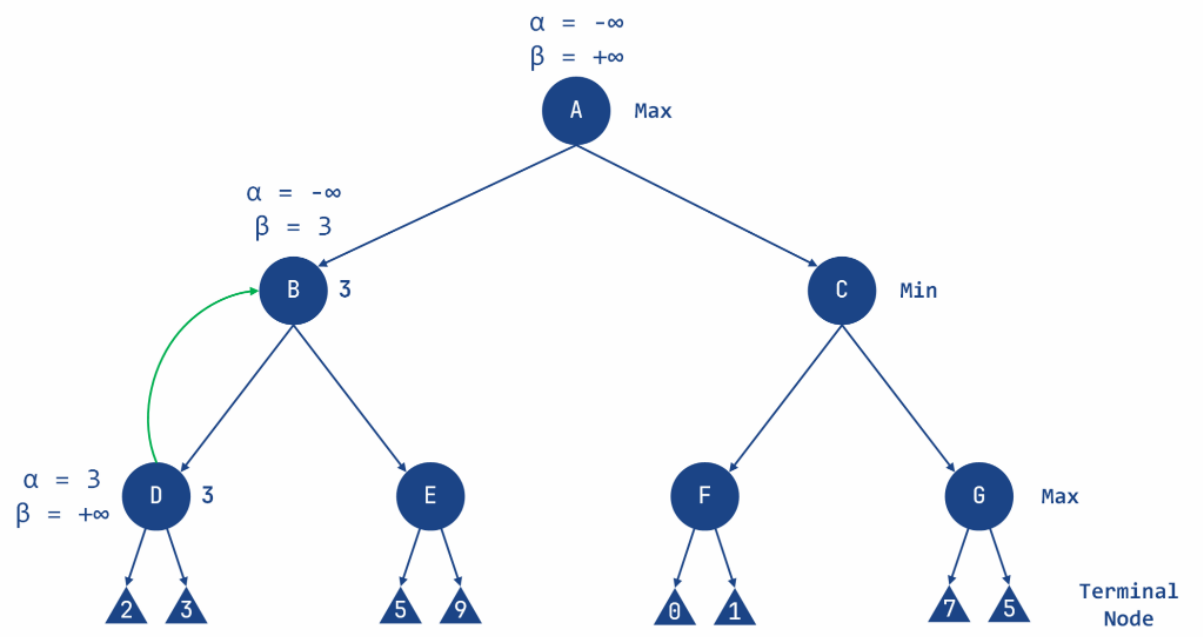


Step 2: Evaluate Node D (`Max's` Turn)

- At **Node D**, it's **`Max's`** turn.
- Compare the node's children values:
  - Left child value = **2**
  - Right child value = **3**
- Update α:
  - α = **`max(2, 3) = 3`**
- Set the value of **Node D** to **3**.

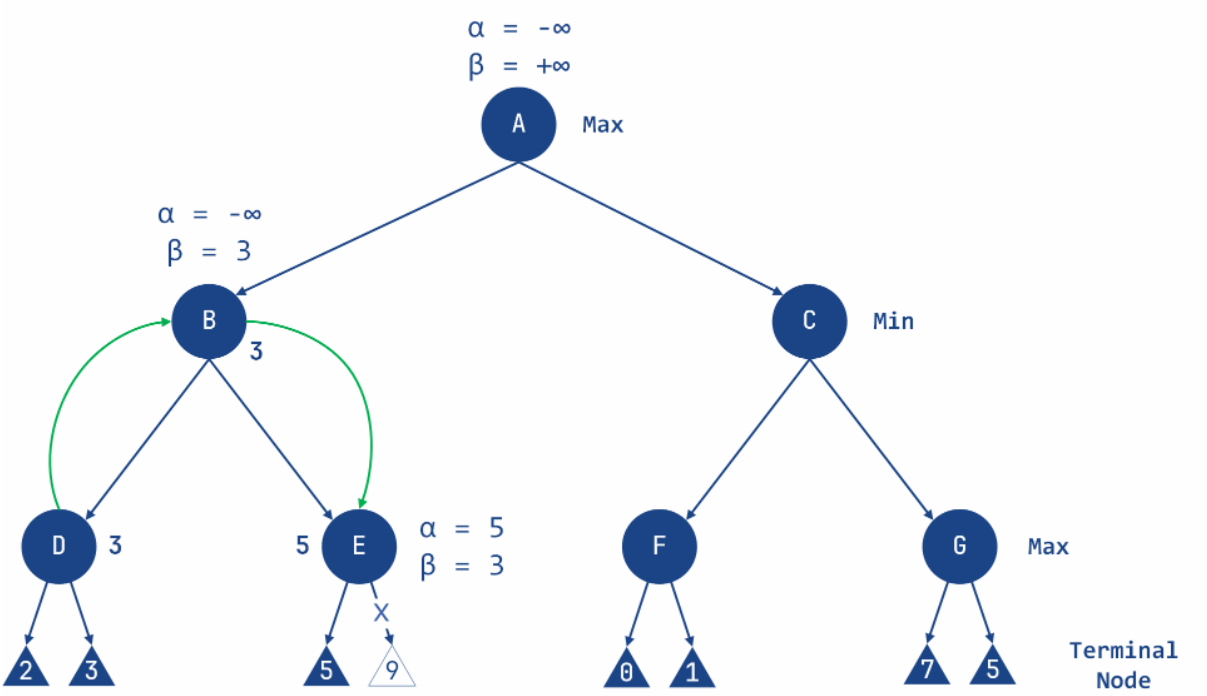
Step 3: Backtrack to Node B (`Min's` Turn)

- Return to **Node B**, where it's **`Min's`** turn.
- Update β:
  - β = **`min(∞, 3) = 3`**
- Now, at **Node B**:
  - α = -∞
  - β = 3
- Pass these values to the next child of **Node B**, which is **Node E**.



Step 4: Evaluate Node E (`Max's Turn`)

- At **Node E**, it's `Max's` turn.
- Compare the node's left child value:
  - Left child value = 5
- Update α:
  - α = `max(-∞, 5) = 5`
- Now, at **Node E**:
  - α = 5
  - β = 3
- Check the pruning condition:
  - Since α >= β, prune the right child of **Node E**.
- Set the value of **Node E** to 5.

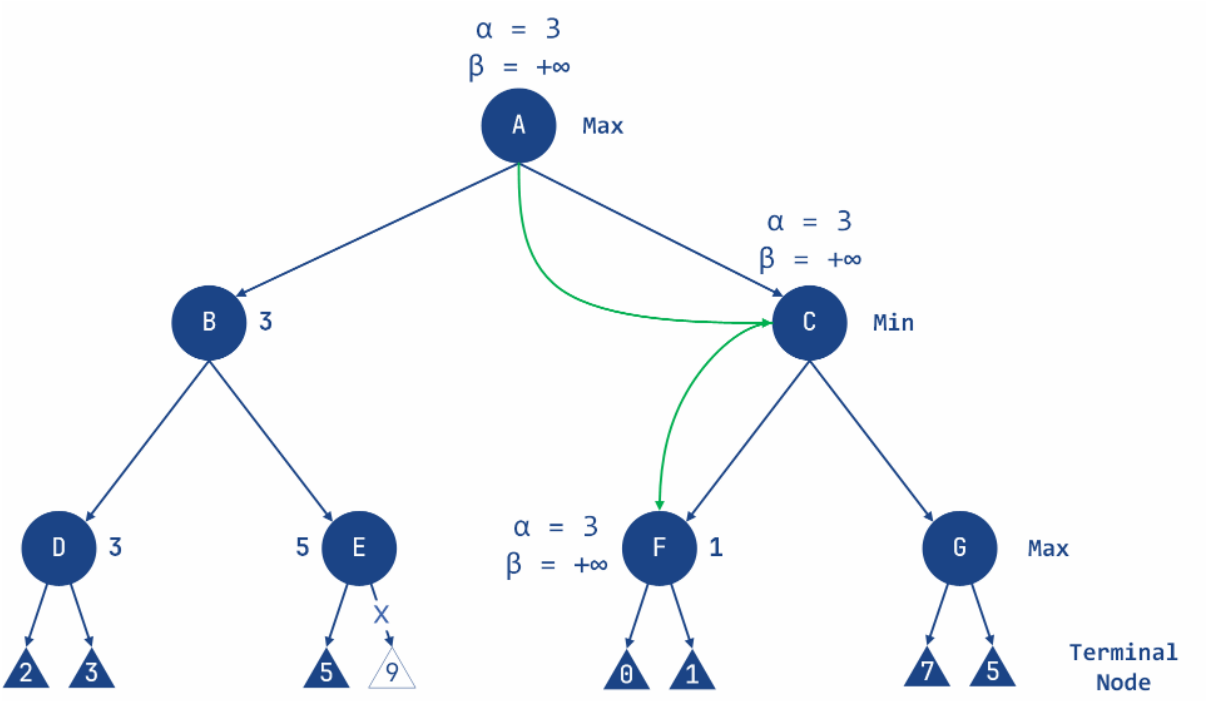


Step 5: Backtrack to Node A (`Max's` Turn)

- Return to **Node A**, where it's **`Max's`** turn.
- Update α:
  - α = **`max(-∞, 3) = 3`**
- Now, at **Node A**:
  - α = 3
  - β = +∞
- Pass these values to the next child of **Node A**, which is **Node C**.

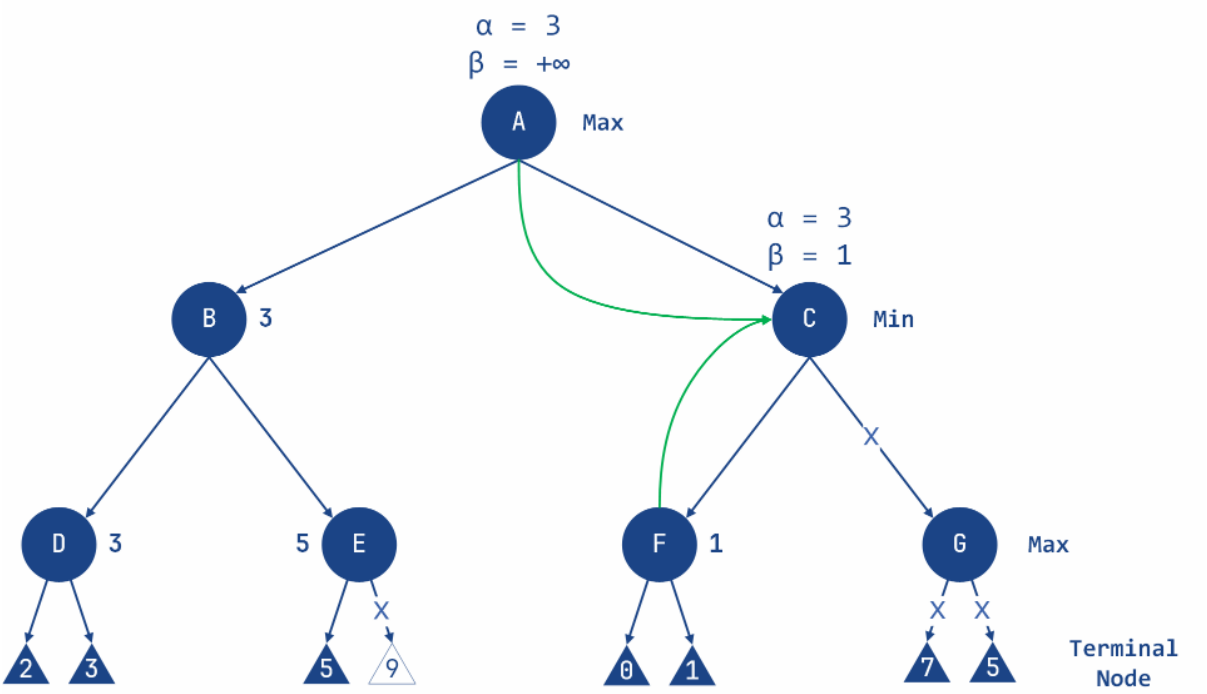
Step 6: Evaluate Node F (`Max's` Turn)

- At **Node F**, it's **`Max's`** turn.
- Compare the node's children values:
  - Left child value = 0
  - Right child value = 1
- Update α:
  - α = **`max(3, 0) = 3`**
  - α = **`max(3, 1) = 3`** (remains unchanged)
- Set the value of **Node F** to 1.



Step 7: Backtrack to Node C (`Min's` Turn)

- Return to **Node C**, where it's **`Min's`** turn.
- Update β:
  - β = **`min(∞, 1) = 1`**
- Now, at **Node C**:
  - α = 3
  - β = 1
- Check the pruning condition:
  - Since α >= β, prune the right child of **Node C** (**Node G**).
- Set the value of **Node C** to 1.

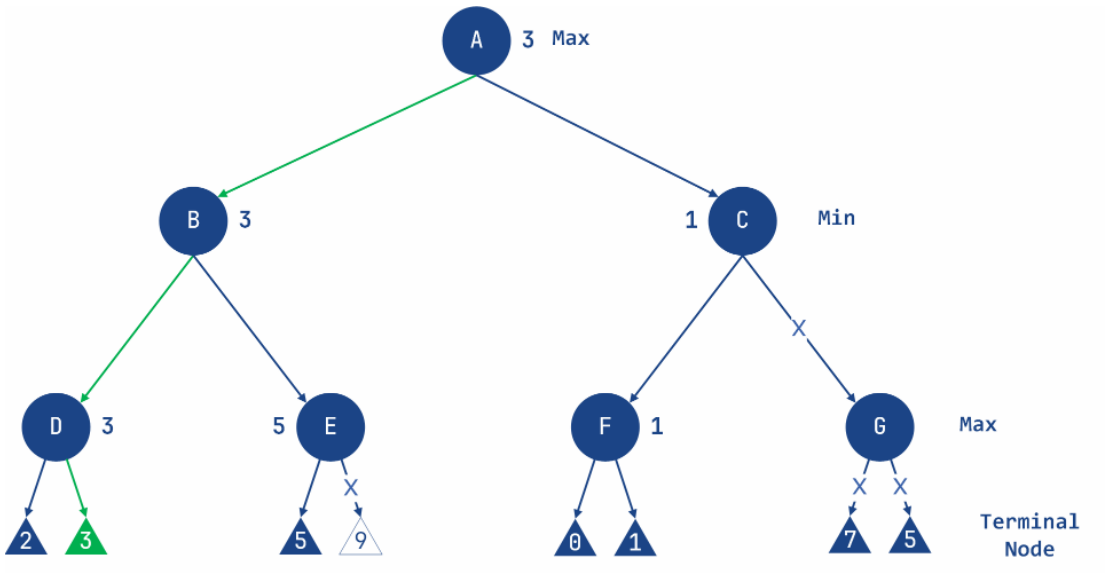


Step 8: Finalize Node A (`Max's` Turn)

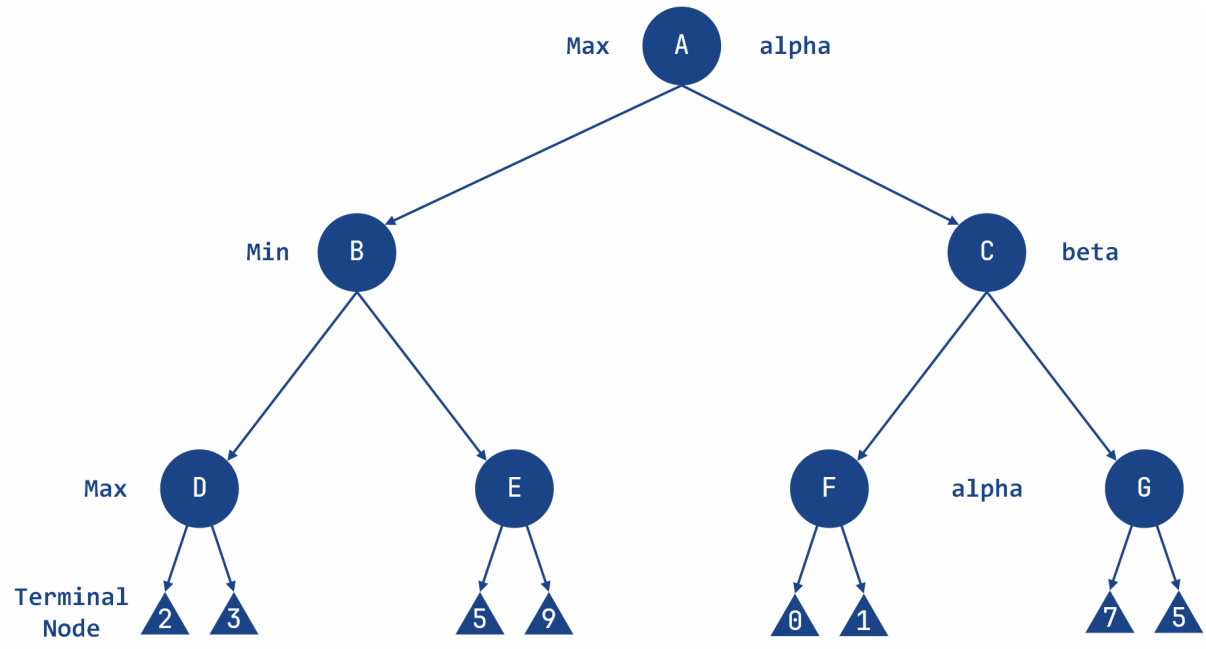
- Return to **Node A**, where it's **`Max's`** turn.
- Update α:
  - α = **`max(3, 1) = 3`**
- The optimal value for the **`Max`** player is **3**.

Final Tree

- Nodes computed: A, B, D, E, C, F
- Nodes pruned: Right child of E, Right child of C (Node G)
- The final tree shows the computed nodes and the pruned nodes. The optimal value for the **`Max`** player is **3**.



2.2.2.3 MiniMax with Alpha-Beta Pruning Algorithm - **Example**




```
import math

class Node:
    def __init__(self, value=None):
        self.value = value
        self.children = []
        self.minmax_value = None
```

```
class MinimaxAgent:
    def __init__(self, depth):
        self.depth = depth

    def formulate_goal(self, node):
        return "Goal reached" if node.minmax_value is not None else
"Searching"

    def act(self, node, environment):
        goal_status = self.formulate_goal(node)
        if goal_status == "Goal reached":
            return f"Minimax value for root node: {node.minmax_value}"
        else:
            return environment.alpha_beta_search(node, self.depth, -
math.inf, math.inf, True)

class Environment:
    def __init__(self, tree):
        self.tree = tree
        self.computed_nodes = []

    def get_percept(self, node):
        return node

    def alpha_beta_search(self, node, depth, alpha, beta,
maximizing_player=True):
        self.computed_nodes.append(node.value)
        if depth == 0 or not node.children:
            return node.value

        if maximizing_player:
            value = -math.inf
            for child in node.children:
                value = max(value, self.alpha_beta_search(child, depth - 1,
alpha, beta, False))
                alpha = max(alpha, value)
                if beta <= alpha:
                    print("Pruned node:", child.value)
                    break
```

```
            node.minmax_value = value
            return value
        else:
            value = math.inf
            for child in node.children:
                value = min(value, self.alpha_beta_search(child, depth - 1,
alpha, beta, True))
                beta = min(beta, value)
                if beta <= alpha:
                    print("Pruned node:", child.value)
                    break
            node.minmax_value = value
            return value

def run_agent(agent, environment, start_node):
    percept = environment.get_percept(start_node)
    agent.act(percept, environment)

# constructing the tree
root = Node('A')
n1 = Node('B')
n2 = Node('C')
root.children = [n1, n2]

n3 = Node('D')
n4 = Node('E')
n5 = Node('F')
n6 = Node('G')
n1.children = [n3, n4]
n2.children = [n5, n6]

n7 = Node(2)
n8 = Node(3)
n9 = Node(5)
n10 = Node(9)
n3.children = [n7, n8]
n4.children = [n9, n10]

n11 = Node(0)
n12 = Node(1)
```

```
n13 = Node(7)
n14 = Node(5)
n5.children = [n11, n12]
n6.children = [n13, n14]

# define depth for Alpha-Beta pruning
depth = 3
agent = MinimaxAgent(depth)
environment = Environment(root)

run_agent(agent, environment, root)

print("Computed Nodes:", environment.computed_nodes)
print("Minimax values:")
print(f"A: {root.minmax_value}")
print(f"B: {n1.minmax_value}")
print(f"C: {n2.minmax_value}")
print(f"D: {n3.minmax_value}")
print(f"E: {n4.minmax_value}")
print(f"F: {n5.minmax_value}")
print(f"G: {n6.minmax_value}")
```

Output

```
Pruned node: 5
Pruned node: F
Nodes computed: ['A', 'B', 'D', 2, 3, 'E', 5, 'C', 'F', 0, 1]
Minimax values:
A: 3
B: 3
C: 1
D: 3
E: 5
F: 1
G: None
```

3. Lab Tasks

3.1. Task 01

Tic-Tac-Toe is a classic two-player game played on a 3×3 grid. The players take turns marking a space with their symbol (**X** or **O**). The goal is to form a straight line of three symbols, either **horizontally, vertically, or diagonally**. If the grid is full and no player has won, the game ends in a draw.

In this task, you will implement an **AI player using the Minimax algorithm**. The AI will analyze the game board and always make the optimal move, ensuring that it **never loses**.

3.1.1. Game Flow

1. **Start of the Game**
   - The board starts empty.
   - The human player is assigned the symbol **O**, and the AI is assigned **X**.
   - The game alternates between the human and AI.
2. **Human Player's Turn**
   - The player enters their move as **row and column indices** (`0-2`).
   - If the move is **valid**, it is placed on the board.
   - If the move is **invalid** (already occupied), the player is asked to try again.
3. **AI's Turn**
   - The AI **calculates the best move** using the Minimax algorithm.
   - The AI places its **X** on the board at the optimal position.
4. **Winning or Drawing Condition**
   - After every move, the program **checks for a winner**.
   - If a player has won, the game announces the **winner**.
   - If the board is full with no winner, the game **ends in a draw**.

```
------------------------------------------------------------
X = AI's Symbol
O = Player Symbol

Positions:
0 0 | 0 1 | 0 2
1 0 | 1 1 | 1 2
2 0 | 2 1 | 2 2
------------------------------------------------------------
                        Sample Output
------------------------------------------------------------

   |   |

   |   |

------------------------------------------------------------
```

```
  |   |
Enter row and column (0-2): 0 0
O |   |
  | X |
  |   |
Enter row and column (0-2): 2 2
O | X |
  | X |
  |   | O
Enter row and column (0-2): 2 1
O | X |
  | X |
X | O | O
Enter row and column (0-2): 0 2
O | X | O
  | X | X
X | O | O
Enter row and column (0-2): 1 0
O | X | O
O | X | X
X | O | O
```

3.2. Task 02:

Imagine a game where two players, **Max and Min**, take turns picking coins from a **row of numbers**. Each player can only pick either **the leftmost or rightmost coin** from the remaining sequence.

Max's Goal: Collect coins with the highest total sum.
Min's Goal: Minimize Max's total sum by making strategic choices.

Since Max moves first, he wants to **find the best sequence of picks** that gives him the highest possible sum. This task uses the **Alpha-Beta Pruning algorithm** to optimize Max's decision-making.

3.2.1. Game Flow

1. **Start of the Game**
   - A list of coins with different values is given (e.g., `[3, 9, 1, 2, 7, 5]`).
   - Max goes **first**, choosing either the **leftmost** or **rightmost** coin.
2. **Player Turns**
   - **Max picks** the best option using **Alpha-Beta Pruning**.
   - The remaining coins are updated.
   - **Min picks** a coin (he chooses the smallest available option to minimize Max's gain).
   - The sequence continues until **no coins remain**.
3. **Winning Condition**
   - Once all coins are taken, the total sum for both players is displayed.
   - The player with the **higher total wins**.

Sample Output

```
Initial Coins: [3, 9, 1, 2, 7, 5]

Max picks 3, Remaining Coins: [9, 1, 2, 7, 5]

Min picks 5, Remaining Coins: [9, 1, 2, 7]

Max picks 9, Remaining Coins: [1, 2, 7]

Min picks 1, Remaining Coins: [2, 7]

Max picks 7, Remaining Coins: [2]

Min picks 2, Remaining Coins: []

Final Scores - Max: 19, Min: 8

Winner: Max
```

3.3. Task 03:

Implement an AI player for chess using the Minimax algorithm with Alpha-Beta Pruning. The AI should be able to:

Evaluate the best possible moves for each piece.
Use Alpha-Beta pruning to reduce unnecessary computations.
Play against a human or another AI.

3.3.1 Game Flow:

1. The board starts with the standard chess setup.
2. Players take turns making moves.
3. The AI should use a depth-limited Minimax algorithm to decide its best move.
4. The game continues until checkmate, stalemate, or a draw is reached.

Additional Challenge: Implement an evaluation function that considers piece values, positional advantages, and possible threats.

3.4. Task 04:

Two players Max and Min take turns selecting letters from a given pool to form a valid word. The goal is to maximize the score by forming the longest valid word. Invalid wo orithm.